

COMPUTER ARCHITECTURE

CSE – 3521

MID-TERM

Mahir Shadid (MAS)

Asst. Lecturer, Dept. of CSE, IIUC

1

SEGMENT - 1

COMPUTER ARCHITECTURE BASICS

- **Computer Architecture:** Design and Organization of computer components to optimize performance.
- **Computer Organization:** How a computer's hardware components work together to execute programs. It includes memory, CPU, and input/output operations.
- **Abacus:** An ancient counting tool used for basic arithmetic operations, considered one of the first computing devices.

COMPUTER ARCHITECTURE BASICS

- **History:** Five generations:

1. First Gen (1940-1956): Computers used Vacuum Tube containing Cathode (electron emission), Anode (electron collection) and Grid (switch).
 - *Example: **ENIAC (Electronic Numerical Integrator and Computer)*** - The first general-purpose electronic computer (built in 1945). It was huge and used vacuum tubes.
2. Second Gen (1956-1963): Computers used Transistors (acted as switch) which were smaller and faster containing Emitter, Base and Collector.
3. Third Gen (1964-1971): ICs (Integrated Circuits) created on Silicon Chips, which were basically “multiple transistors”.
4. Fourth Gen (1971-present): Microprocessors (thousands of transistors), contains CPU, Memory Units etc. Also known as “system on chips”.
5. Fifth Gen (Future): AI and quantum computing.

COMPUTER ARCHITECTURE BASICS

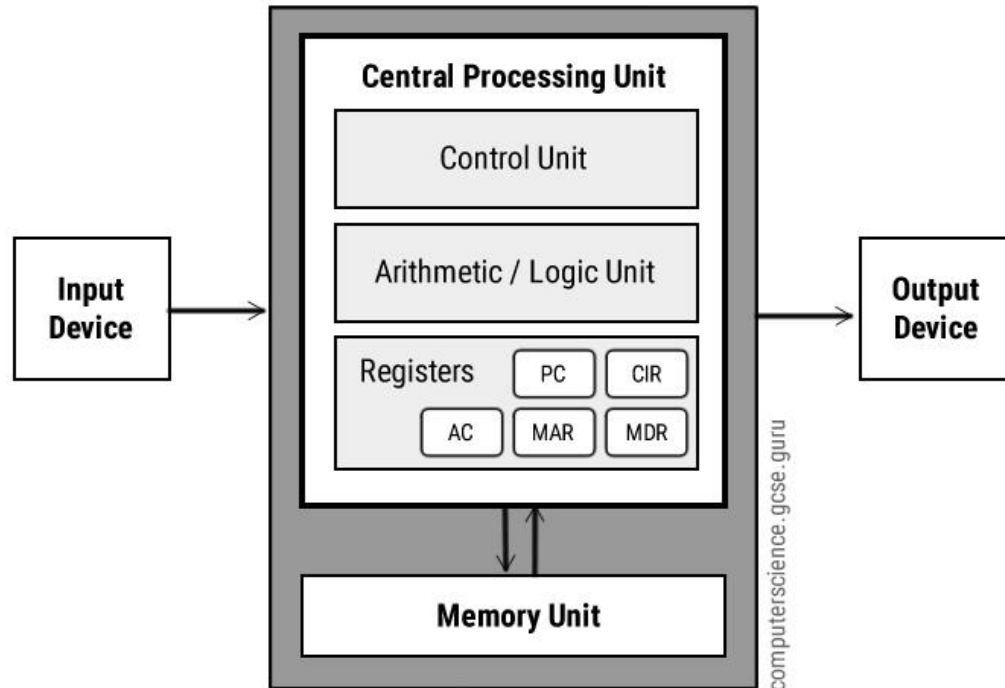
- **Types of Computer Architecture:**

1. Von Neumann (Recent computers).
2. Harvard (embedded-systems).
3. RISC (Reduced Instruction Set Computer): used in ARM-based devices.
4. CISC (Complex Instruction Set Computer): used in x86-based devices.

- **Von Neumann Architecture: Components:**

1. CPU (Central Processing Unit): An electronic circuit (microprocessor) that executes instructions.
2. Control Unit: Controls the operations of ALU (Arithmetic Logic Unit), memory and I/O. Manages data flow.
3. Registers (High speed storage cells).
4. Memory Unit: primarily consists of RAM.

VON NEUMANN ARCHITECTURE



PC (Program counter): holds the address of next instruction.

AC (Accumulator): ALU operations occur here.

MAR (Mem. Address Register): stores the address of memory to be accessed (either for fetching an instruction or reading/writing data).

MDR (Mem. Data Register): stores the data of an instruction.

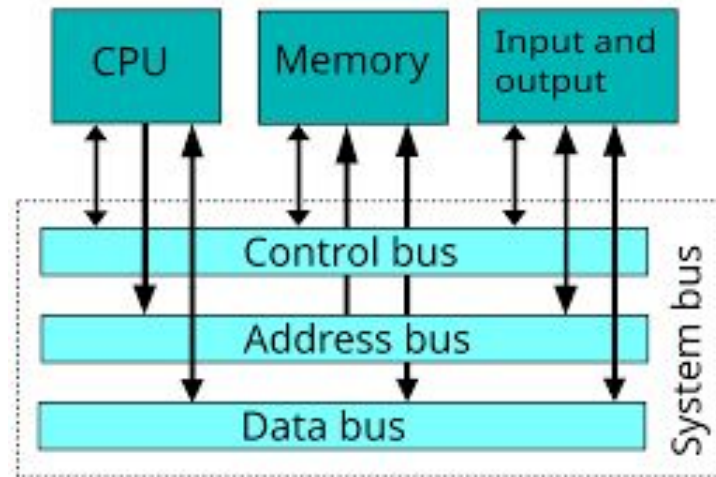
CIR (Current Instruction Register): holds the current instruction.

VON NEUMANN ARCHITECTURE

- Advantages of Von Neumann:
 1. **Simple & Cost-Effective:** Uses a single memory for both data & instructions, making it easier and cheaper to build.
 2. **Flexible:** Can execute different programs by just loading new instructions into memory.
 3. **Efficient Use of Memory:** Same memory can store both code and data dynamically.
 4. **Widespread Usage:** Most modern computers use this design, making it well-supported.
- Disadvantages of Von Neumann Architecture:
 1. **Von Neumann Bottleneck:** The single memory bus limits the speed, as instructions and data share the same path.
 2. **Slower Execution:** Fetching data and instructions sequentially can slow down performance.
 3. **Risk of Overwriting:** Since instructions and data share the same memory, a bug can accidentally modify the program.

SYSTEM BUS

- For connecting the components in a computer architecture we use a system **bus** (set of parallel wires). System Bus consists of:
 1. Data Bus: bi-directional (two-way data flow), used to carry data and instruction. Transmits data between CPU, Memory and I/O.
 2. Address Bus: unidirectional (one-way), carries memory location where data needs to be sent. Carries address from CPU to RAM and I/O.
 3. Control Bus: bi-directional, used to carry control signals (Read, write, interrupt, reset).



RISC (REDUCED INSTRUCTION SET COMPUTER) ARCHITECTURE

- Small, highly optimized instruction set that executes in a single clock cycle. Instructions set in RISC are fixed-length (usually 32-bits). RISC enables pipelining (executing multiple instructions simultaneously in different stages of the instruction cycle).
- *Example:* Multiplying 2 values:
 1. LOAD R1, A ; Load value A into register R1
 2. LOAD R2, B ; Load value B into register R2
 3. MUL R3, R1, R2 ; Multiply R1 and R2, store in R3
 4. STORE R3, C ; Store result in memory location C
- 1. **Few, simple instructions:** Each instruction takes one clock cycle.
- 2. **Fixed instruction size:** Easy to decode and execute.
- 3. **More registers:** Uses registers for computations instead of memory.
- 4. **More lines of code:** Simple instructions mean more instructions are needed to complete a task.
- 5. **Less complex hardware:** Easier to design and optimize.
- 6. **Used in modern processors:** ARM, MIPS.

CISC (COMPLEX INSTRUCTION SET COMPUTER) ARCHITECTURE:

- Single but complex instruction set, can perform multiple operations, can take multiple clock cycles, has variable-length (8-120 bits).
- *Example:* Multiplying 2 values:
 1. MUL A, B, C ; Multiply values at memory locations B and C, store in A
- 1. **Many complex instructions:** One instruction can do multiple tasks.
- 2. **Variable instruction size:** Harder to decode and execute.
- 3. **More memory operations:** Relies more on RAM instead of registers.
- 4. **Fewer lines of code:** One complex instruction does more work.
- 5. **More complex hardware:** Harder to design but powerful.
- 6. **Used in traditional and desktop processors:** Intel x86, AMD.

UNDERSTANDING PROGRAM PERFORMANCE

- **Program performance:** ability of a computer program to execute programs efficiently. Measured in execution time and throughput.
- **Execution time:** time taken to complete a task. Lower, Better.
- Execution time
 1. = CPU clock cycles for a program (CPI) * clock cycle time (*Using Clock Cycle Time*)
 2. = CPU clock cycles for a program / Clock rate (*General Formula*)
 3. = Instruction Count x CPI / Clock Rate (*Using Instruction Count & CPI*)
- **Throughput:** number of tasks completed in a given time. Higher, Better.
- **Speed up ratio:** execution time on slow device/execution time on fast device.

UNDERSTANDING PROGRAM PERFORMANCE

- **CPU clock cycle:** It determines how fast the processor executes instructions.
- **Response time:** The total time from when a user gives an input until the system gives the output.
- **CPU Time:** The actual time the CPU spends processing a program (excluding time spent waiting for input/output).
- **System CPU Time:** The time the CPU spends on system-level operations (like managing memory, handling I/O), not just user programs.
- **Relative Performance:** A comparison of how fast one computer or processor is compared to another. Higher relative performance means a faster system.
- **CPI (Cycles Per Instruction):** The average number of clock cycles needed to execute one instruction. Lower CPI means faster performance.

UNDERSTANDING PROGRAM PERFORMANCE

- **Key factors** that affects computer performance:
 1. Clock Speed (expressed in GHz, instructions processed per second) - 4GHz performs better than 3GHz.
 2. Instruction count (fewer, faster).
 3. CPI (how many cycles a CPU takes per instruction).

AMDAHL'S LAW

- Determines maximum possible performance gain by improving one part of the system.
- Calculated by, $\text{Speed Up} = 1 / (1-p) + (p/s)$
 - P = fraction of improved task
 - S = speed up factor / Number of processors
- *Example:* if 50% of a task is optimized, and speed up is 2x, overall speed up would be,

$$1 / (1-0.5) + (0.5/2) = 1.33x$$

MIPS (MILLIONS INSTRUCTIONS PER SECOND) - 32 BITS

- A metric used to measure the performance of a computer's processor. It refers to how many millions of instructions a processor can execute in one second.
1. $\text{MIPS} = \text{Instruction Count} / \text{Execution time} \times 10^6$
 2. $\text{MIPS} = \text{Clock rate} / \text{CPI} \times 10^6$
 3. $\text{MIPS} = \text{Total Instruction} / \text{Execution Time}$

Example:

Device	Instruction Counts in Billions		
	A	B	C
Compiler 1	4	2	3
Compiler 2	10	5	4

MIPS (MILLIONS INSTRUCTIONS PER SECOND) - 32 BITS

Device	Instruction Counts in Billions		
	A	B	C
Compiler 1	4	2	3
Compiler 2	10	5	4

Instruction Class	CPI
A	1
B	2
C	3

If the computer's clock rate is 5 GHz, which code sequence will execute faster according to execution time?

MIPS (MILLIONS INSTRUCTIONS PER SECOND) - 32 BITS

Total Cycles (Total CPI) = $\sum(\text{Instruction count} \times \text{Cycles per instruction})$

1. Compiler 1's total cycle = $(4 \times 1) + (2 \times 2) + (3 \times 3) = 17$ billion cycles
2. Compiler 2's total cycle = $(10 \times 1) + (5 \times 2) + (4 \times 3) = 32$ billion cycles

Execution time = Total cycle / clock rate

3. Compiler 1's execution time = $17 / 5 = 3.4$ s. (5 GHz is given)
4. Compiler 2's execution time = $32 / 5 = 6.4$ s. (5 GHz is given)

MIPS = Total instruction / execution time

5. Compiler 1's MIPS = $(4+2+3) \text{ 9000 millions instructions} / 3.4\text{s} = 9000 / 3.4 = 2647$ MIPS.
6. Compiler 2's MIPS = $(10+5+4) \text{ 19000 millions instructions} / 6.4\text{s} = 19000 / 6.4 = 2969$ MIPS.

Ans: This shows Compiler 2 has a higher MIPS but slower execution time, a classic pitfall of MIPS as a performance metric.

MIPS (MILLIONS INSTRUCTIONS PER SECOND) - 32 BITS

- Now try:
 - Suppose we have two implementations of the same instruction set architecture.
Computer1 has a CPI of 3.0 and a clock cycle time of 300 ps for some programs, and
Computer2 has a CPI of 4.0 and a clock cycle time of 150 ps for the same programs.
Which computer is faster for this program and by how much?

MIPS (MILLIONS INSTRUCTIONS PER SECOND) - 32 BITS

- Suppose we have two implementations of the same instruction set architecture. Computer1 has a CPI of 3.0 and a clock cycle time of 300 ps for some programs, and Computer2 has a CPI of 4.0 and a clock cycle time of 150 ps for the same programs. Which computer is faster for this program and by how much?
- Which one is faster?
 1. Execution time = CPI x Clock cycle time
 2. Computer1 = $3 \times 300 = 900\text{ps}$
 3. Computer2 = $4 \times 150 = 600\text{ps}$ (Faster)
- How much faster?
 1. Speed up ratio = execution time of slow device / execution time of fast device = $900/600 = 1.5$.
 2. Which means 1.5x faster.

MIPS (MILLIONS INSTRUCTIONS PER SECOND) - 32 BITS

- What is CPU clock cycle and clock rate? UNIX time command is given **40.4u 10.5s 2:50**. Calculate the percentage according to total elapsed time.
- **Given:**
 1. User CPU time = **40.4 s**
 2. System CPU time = **10.5 s**
 3. Total elapsed time = **2:50** = $2 \times 60 + 50 = 170$ seconds
- The question says *percentage according to total elapsed time*. Usually, this means:

$$\text{Percentage} = \frac{\text{Total CPU Time (User and System)}}{\text{Total Elapsed Time}} \times 100$$

- Total CPU time = $40.4 + 10.5 = 50.9$ s

MIPS (MILLIONS INSTRUCTIONS PER SECOND) - 32 BITS

▪

$$\text{Percentage} = \frac{50.9}{170} \times 100 = 29.94\% \text{ (ANS)}$$

- MIPS does not always directly indicate performance because:
 1. **Different CPI values:** A machine with a high MIPS value may have a low CPI, but it does not necessarily mean it completes tasks faster.
 2. **Instruction Set Differences:** Some architectures may execute instructions in fewer cycles but at a lower MIPS rate.
 3. **Clock Speed Impact:** A higher clock speed can lead to a higher MIPS, but actual execution time may still be longer.

Thus, a machine with higher MIPS does not always execute a program faster if CPI is also higher.

SEGMENT-2

INSTRUCTION SET ARCHITECTURE

- **ISA** (Instruction Set Architecture): Defines the types of instructions supported by a processor.
- Types:
 1. Stack-based ISA: Uses a stack for all operations (push/pop). No direct access to registers. Example: JVM (Java Virtual Machine).
 2. Accumulator-based ISA: Uses a single accumulator register for most operations. Simple but results in more memory access. Example: Intel 8008, 8051 Microcontrollers.
 3. Register-based ISA: Uses multiple registers for operations instead of a stack or accumulator. Faster than stack or accumulator-based architectures. Example: Intel x86.

INSTRUCTION SET ARCHITECTURE

- **Instruction Set:** Collection of machine language instruction. 5 types:
 1. Arithmetic: add, sub, mul, div.
 2. Logical: AND, OR, NOT.
 3. Data Transfer: MOV, LOAD, STORE.
 4. Control Flow: JUMP, CALL, RETURN.
 5. I/O

INSTRUCTION SET ARCHITECTURE

- **Instruction Access methods:** Instruction access refers to how the CPU fetches and retrieves instructions from memory before executing them. The CPU follows the fetch-decode-execute cycle to process each instruction.
 1. **Sequential Access:** CPU fetches instructions one after another in a sequence. After executing an instruction, the PC is incremented to point to the next instruction.
 2. **Direct Access:** The instruction address is directly specified in the instruction itself. Used in branching and jumping instructions. *Example:* JMP 0x200 ; Jump to memory location 0x200.
 3. **Indirect Access:** The instruction contains an address that points to another memory location where the actual instruction is stored. *Example:* JMP [0x200] ; Fetch address from location 0x200 and jump there.

INSTRUCTION SET ARCHITECTURE

- **Data Access Methods:** Data access refers to how the CPU retrieves and stores data in memory or registers during instruction execution. *Different addressing modes* determine how the data is accessed.
 1. **Register Addressing:** data (operand) is stored in a CPU register. *Ex:* ADD R1, R2, R3 ; $R1 = R2 + R3$.
 2. **Immediate Addressing:** data (operand) is part of the instruction itself. *Ex:* ADDI R1, R2, 10 ; $R1 = R2 + 10$.
 3. **Direct Addressing:** The instruction specifies the exact memory address where the operand is located. *Ex:* LOAD R1, 0x2000 ; Load value from memory address 0x2000 into R1.
 4. **Indirect Addressing:** The instruction provides a memory location that stores another address where the actual operand is located. *Ex:* LOAD R1, [0x2000] ; Fetch address stored at 0x2000, then load value from that address.
 5. **Indexed Addressing:** A base address is combined with an offset stored in an index register. *Ex:* LOAD R1, 100(R2) ; Load value from memory address $(R2 + 100)$ into R1.

STORED-PROGRAM CONCEPT

- The Stored-Program Concept is a fundamental principle of modern computing, introduced by John von Neumann. This concept is the basis for the *Von Neumann Architecture*, where instructions and data are stored in the same memory and accessed via the same data bus. Programs can be modified during execution.
- Consider a simple program that **adds two numbers** stored in memory:
- Step-by-Step Execution:
 1. Program and data are loaded into memory: CPU fetches both instructions and operands from RAM.
 2. CPU Fetches and Decodes the Instruction: The first instruction (e.g., LOAD A) is fetched and sent to the Instruction Register (IR). The Program Counter (PC) is updated to point to the next instruction.
 3. Instruction Execution: The instruction is executed (e.g., the addition is performed in the ALU).
 4. Store Result in Memory: The result is stored back in RAM for later use.

OPERATIONS OF THE COMPUTER HARDWARE

- Computer hardware operations follow the fetch-decode-execute cycle:
 1. Fetch: The CPU retrieves an instruction from memory.
 2. Decode: The instruction is interpreted by the control unit.
 3. Execute: The ALU (Arithmetic Logic Unit) or other components perform the operation.
 4. Write Back: The result is stored in a register or memory.

OPERANDS OF THE COMPUTER HARDWARE

- Operands are the values or data on which instructions operate. They can be stored in:
 1. Registers: Fastest storage location within the CPU.
 2. Memory: Slower access, requires address calculation.
 3. I/O Devices: External inputs such as keyboard, mouse, or sensors.

- Design Principles of Computer Hardware:
 1. Simplicity Favors Regularity: Using uniform instruction formats.
 2. Smaller is Faster: Reducing instruction complexity speeds up execution.
 3. Make the Common Case Fast: Optimizing frequent operations.

REPRESENTATION OF INSTRUCTIONS IN THE COMPUTER

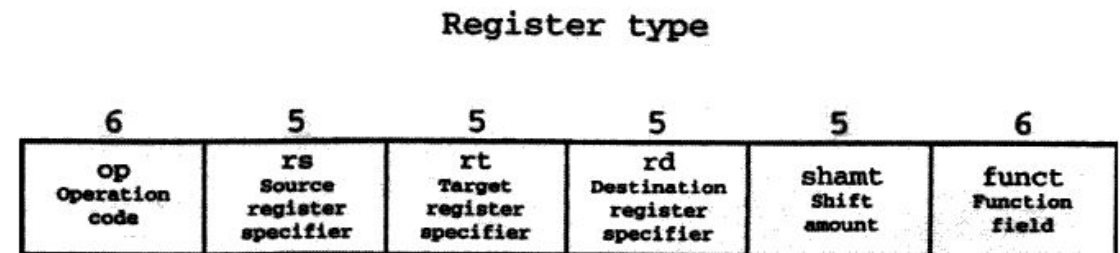
- Instructions are stored in binary format. The MIPS instruction format consists of:
 1. Opcode: Specifies the operation (ADD, LOAD, etc).
 2. Operands: Register/memory locations involved.
 3. Addressing Mode: Defines how to locate data.
- Instructions for **Decision Making**: Conditional and branching instructions alter the program flow based on conditions.
 1. Branching:
 1. BEQ (Branch if Equal): Jumps to a label if two registers are equal.
 2. BNE (Branch if Not Equal): Jumps if they are different.
 2. Jumping:
 1. J (Jump): Unconditionally jumps to a new instruction address.

MIPS ADDRESSING FOR 32-BIT IMMEDIATE AND ADDRESSES

- MIPS uses different addressing modes for memory access and instruction execution.
 1. Immediate Addressing: The operand is directly in the instruction.
 2. Register Addressing: Uses register values.
 3. Base (Displacement) Addressing: Adds an immediate offset to a base register.
 4. PC-Relative Addressing: Used in branch instructions. The offset is added to the program counter.
 5. Jump Addressing: Used in J and JAL instructions to specify an absolute jump address.

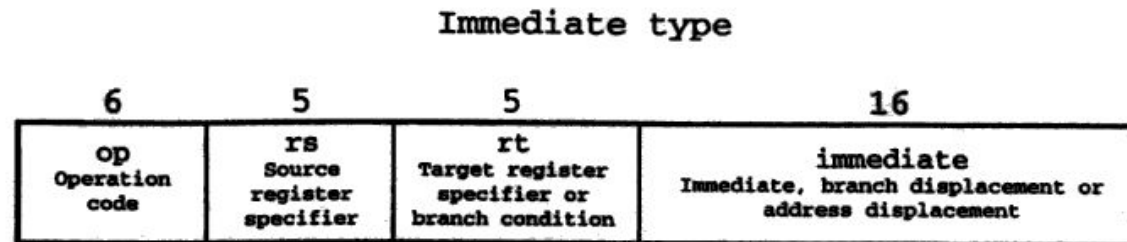
INSTRUCTION FORMATS/MIPS ADDRESSING MODES

- **Instruction formats** define the structure of machine-level instructions, specifying how operands, opcodes, and addressing modes are arranged. There are three primary instruction formats:
- **R-Type (Register-Type) Format:** Used for register-to-register operations (mostly arithmetic and logic operations).
 - **Fields** (Total 32 bits): opcode (6 bits) | rs (5 bits) | rt (5 bits) | rd (5 bits) | shamt (5 bits) | funct (6 bits)
 1. Opcode (6 bits) – Operation code
 2. rs (5 bits) – Source register
 3. rt (5 bits) – Target register
 4. rd (5 bits) – Destination register
 5. shamt (5 bits) – Shift amount (used for shift operations like sll, srl, etc.)
 6. funct (6 bits) – Function code for ALU operations



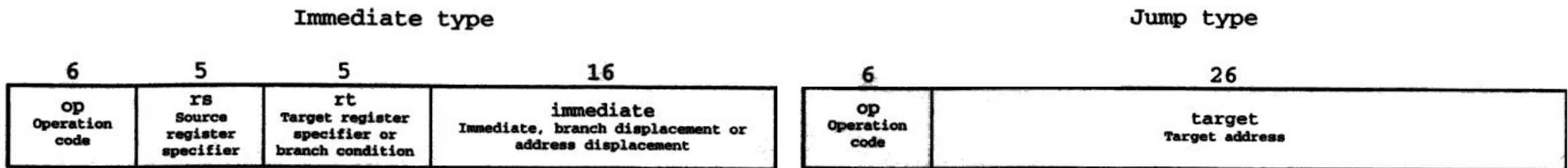
INSTRUCTION FORMATS/MIPS ADDRESSING MODES

- **I-Type (Immediate-Type) Format:** Used for operations with an immediate value or memory access instructions. Immediate values or memory addresses are directly provided in the instruction.
 - **Fields** (32 bits): opcode (6 bits) | rs (5 bits) | rt (5 bits) | offset (16 bits)
 1. Opcode (6 bits) – Operation code
 2. rs (5 bits) – Source register
 3. rt (5 bits) – Destination register
 4. Immediate (16 bits) – Constant or memory offset



INSTRUCTION FORMATS/MIPS ADDRESSING MODES

- **J-Type (Jump-Type) Format:** Used for unconditional jumps (changing execution flow). Contains only an address to jump to.
 - Fields (32 bits): Opcode (6 bits) | Target Address (26 bits)
 1. Opcode (6 bits) – Operation code
 2. Target Address (26 bits) – Jump destination
- Two types of Opcode:
 1. j → Jump 2 steps ahead → 000010
 2. jal → Jump + Link (1 extra) → 000011



MIPS REGISTER MAPPING

- An easy way to remember the MIPS register numbers in a structured way:
 1. The temporary registers (\$t0 to \$t9) start at 8 and go up to 15 (\$t0 = 8, \$t1 = 9, ..., \$t7 = 15), then \$t8 is 24 and \$t9 is 25.
 2. The saved registers (\$s0 to \$s7) start at 16 and go up to 23 (\$s0 = 16, \$s1 = 17, ..., \$s7 = 23).
 3. *The special purpose registers include \$zero (0), \$at (1), \$v0 (2), \$v1 (3), and the argument registers \$a0 to \$a3 (4 to 7).*
 4. *The kernel registers \$k0 and \$k1 are 26 and 27, while \$gp (global pointer) is 28, \$sp (stack pointer) is 29, \$fp (frame pointer) is 30, and \$ra (return address) is 31.*

MIPS REGISTER MAPPING

Register	MIPS register numbers
Temporary Registers (\$t0 - \$t7)	8 – 15
Temporary Register \$t8	24
Temporary Register \$t9	25
Saved Registers \$s0 - \$s7	16 – 23
Argument Registers \$a0 - \$a3	4 - 7

MIPS REGISTER MAPPING: FUNCTION CODES

- Addition (add = 100000 = 0x20): Think of "20 means add", since it's a basic operation.
- Subtraction (sub = 100010 = 0x22): It's just add but with the 2nd bit flipped (10).
- Logical operations (and, or, xor, nor):
 - 1001XX → and, or, xor, nor follow a pattern.
 - and = 100100, or = 100101, xor = 100110, nor = 100111 (incrementing last 2 bits).
- Set Less Than (slt = 101010):
 - Think of "10-10" (slt checks if one number is less than another).
- Branching:
 1. Beq = 000100
 2. Bne = 000101

MIPS REGISTER MAPPING: OPERATION CODES

Instruction	Type	Opcode (binary)
add	R	000000
sub	R	000000
slt	R	000000
addi	I	001000
lw	I	100011
sw	I	101011
beq	I	000100
bne	I	000101
j	J	000010

MIPS REGISTER MAPPING: EXAMPLES

- Example:

Translate the following instruction into machine instruction:

Add \$t5, \$s3, \$s4

- This is a R-type instruction. Thus:

1. Add = 100000 (Function codes are in 6 digits binary)
2. T5 = 13 = 01101 (Register mappings are 5 digits)
3. S3 = 19 = 10011
4. S4 = 20 = 10100

- Machine instruction for R-type

1. = opcode (6 bits) | rs (5 bits) | rt (5 bits) | rd (5 bits) | shamt (5 bits) | funct (6 bits)
2. = 000000 (for add) | s3 | s4 | t5 | 00000 (for add) | 100000 (func code of add)
3. = 000000 10011 10100 01101 00000 100000
4. = 0x027A3408 Hex (Ans.)

MIPS REGISTER MAPPING: EXAMPLES

`beq $t1, $s2, 1000` (I-type problem because of direct constant)

- So structure is “opcode (6 bits) | rs (5 bits) | rt (5 bits) | offset (16 bits)”
 1. Beq = 000100
 2. T1 = 9 = 01001
 3. s2 = 18 = 10010
 4. 1000 = 0000001111101000
- Machine instruction for I-type = 000100 01001 10010 0000001111101000 = 0x112407D0 Hex (Ans.)

MIPS REGISTER MAPPING: EXAMPLES

SLL (Shift Left Logical) - R-type Instruction: `sll $t0, $t1, 5`

- Field - opcode (6) | rs (5) | rt (5) | rd (5) | shamt (5) | funct (6)
 1. opcode -> Operation code (always 0 for R-type) -> 000000
 2. Rs -> Source register (Not used in SLL) -> 00000
 3. Rt -> Second source register -> \$t1 -> 01001
 4. Rd -> Destination register -> \$t0 -> 01000
 5. Shamt -> Shift amount -> 5 -> 00101
 6. Funct -> Function code for SLL -> 000000
- Binary Code: 000000 00000 01001 01000 00101 000000
- HEX Code: 0x00094880 (Ans.)

SEGMENT-3

ARITHMETIC AND LOGICAL OPERATIONS

- **Arithmetic Operations:**

- Addition: Adds two numbers together. **Example:** $5 + 10 = 15$
- Subtraction: Subtracts one number from another. **Example:** $15 - 5 = 10$

- **Logical Operations:**

- Bitwise AND (&): Sets each bit to 1 if both bits are 1. **Example:** $5 \& 3 \rightarrow 0101 \& 0011 = 0001$ (Result: 1)
- Bitwise OR (|): Sets each bit to 1 if one of the bits is 1. **Example:** $5 | 3 \rightarrow 0101 | 0011 = 0111$ (Result: 7)

SIGNED AND UNSIGNED NUMBERS

- Unsigned numbers can only represent positive values, while signed numbers can represent both positive and negative values.
- **Example:** Representing -5 in 4-bit Two's Complement
 1. Binary of +5: 0101
 2. One's Complement: Invert all bits $\rightarrow$ 1010
 3. Two's Complement: Add 1 $\rightarrow$ 1011
 4. Signed interpretation: -5, Unsigned interpretation: 11

NUMBER CONVERSION AND REPRESENTATION

- Conversion between different numerical systems (binary, decimal, hexadecimal) and representation in computer memory.

- **Example:** Converting 13 from Decimal to Binary:

Division by 2 Method:

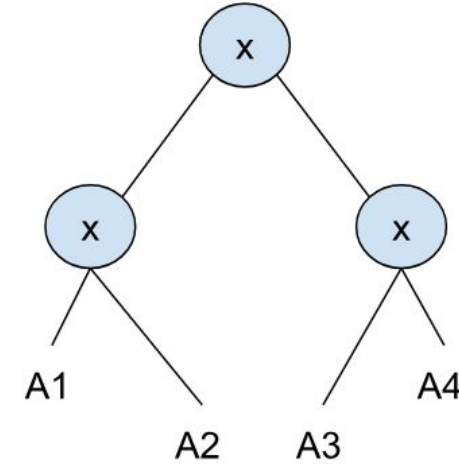
1. $13 \div 2 = 6$, remainder = 1
2. $6 \div 2 = 3$, remainder = 0
3. $3 \div 2 = 1$, remainder = 1
4. $1 \div 2 = 0$, remainder = 1
5. Read remainders bottom to top:
6. 13 (Decimal) = 1101 (Binary)

MATRIX-CHAIN MULTIPLICATION

- A problem in which the goal is to find the most efficient way to multiply a sequence of matrices.
- Conditions:
- Suppose you have two matrices **A** and **B** with **rows * columns** are **5*4** and **4*3**, if the first matrix (A)'s column (4) is equal to the second matrix(B)'s row (4), then we can multiply them.
 1. By following it, if **A * B = C**, then C's dimension will be **5*3**, which means row of **A * column of B**.
 2. So how many multiplications do we need to fill up the matrix C?
 3. Cost = $5*3*4 = 60$ multiplications.
- How many ways can we multiply some matrices?

MATRIX-CHAIN MULTIPLICATION

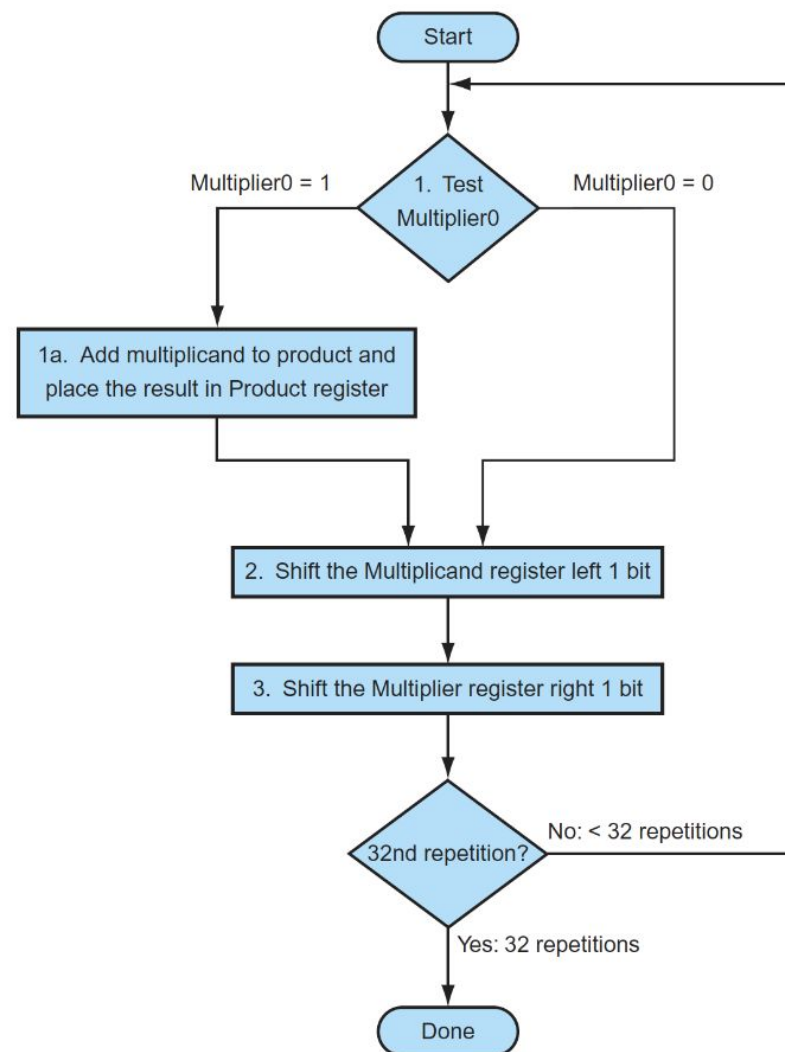
- Suppose we have $(A1 \times A2) \times (A3 \times A4)$
- The tree would be:



- Simple equation to take out the possible ways:
- where n is the number of nodes.
- $= (2n! / n!n!) / n+1$

$$T(n) = \frac{2^n C_n}{n+1}$$

FIRST VERSION OF MULTIPLICATION ALGORITHM



Multiply 3 x 2 ?

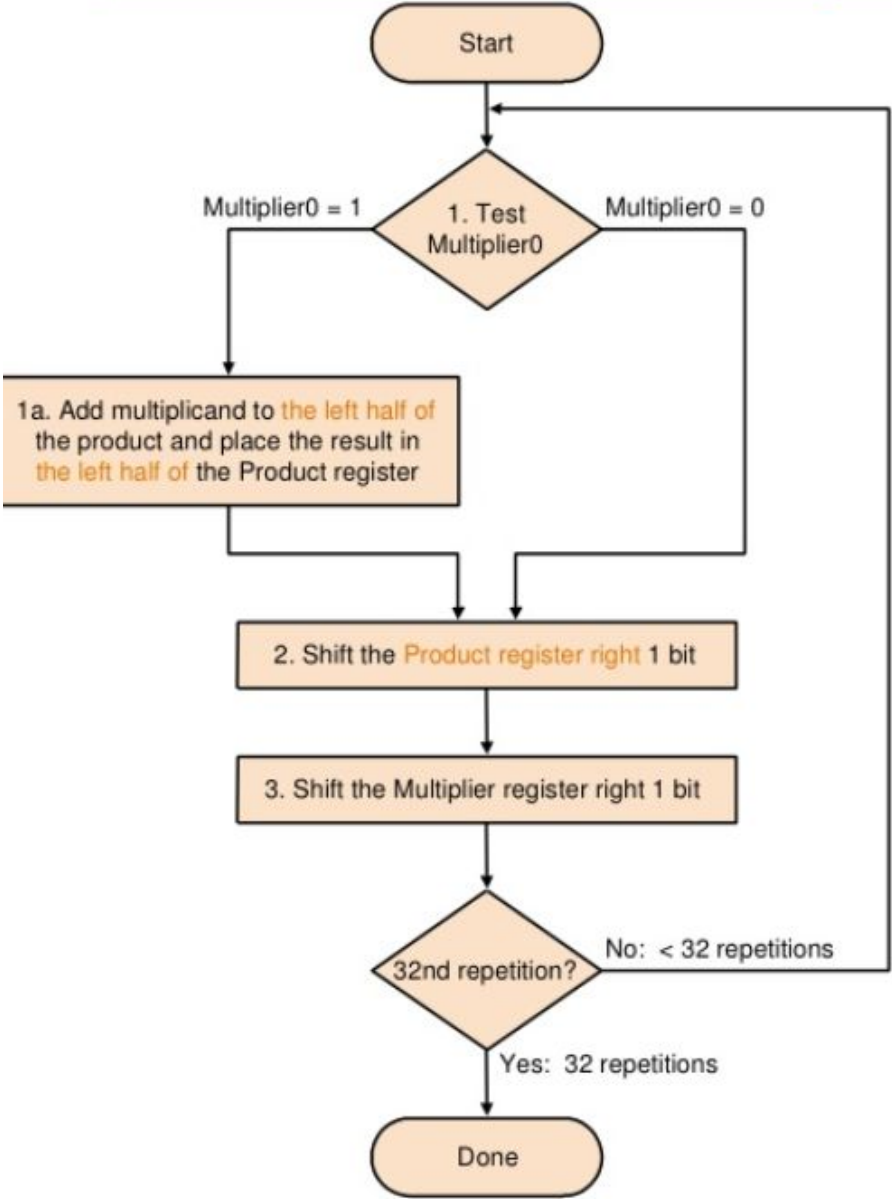
2 = 0010

3 = 0011

Steps	Multiplicand	Multiplier	Product
Init	0011	0010 0010	0000 0000
2	0110	0010 0010	0000 0000
3	0110	0010 0001	0000 0000
1a	0110	0010 0001	0000 0110
2	1100	0010 0001	0000 0110
3	1100	0010 0000	0000 0110
			110 = 6 (ANS)

FIGURE 3.4 The first multiplication algorithm, using the hardware shown in Figure 3.3. If the least significant bit of the multiplier is 1, add the multiplicand to the product. If not, go to the next step. Shift the multiplicand left and the multiplier right in the next two steps. These three steps are repeated 32 times.

SECOND VERSION OF MULTIPLICATION ALGORITHM



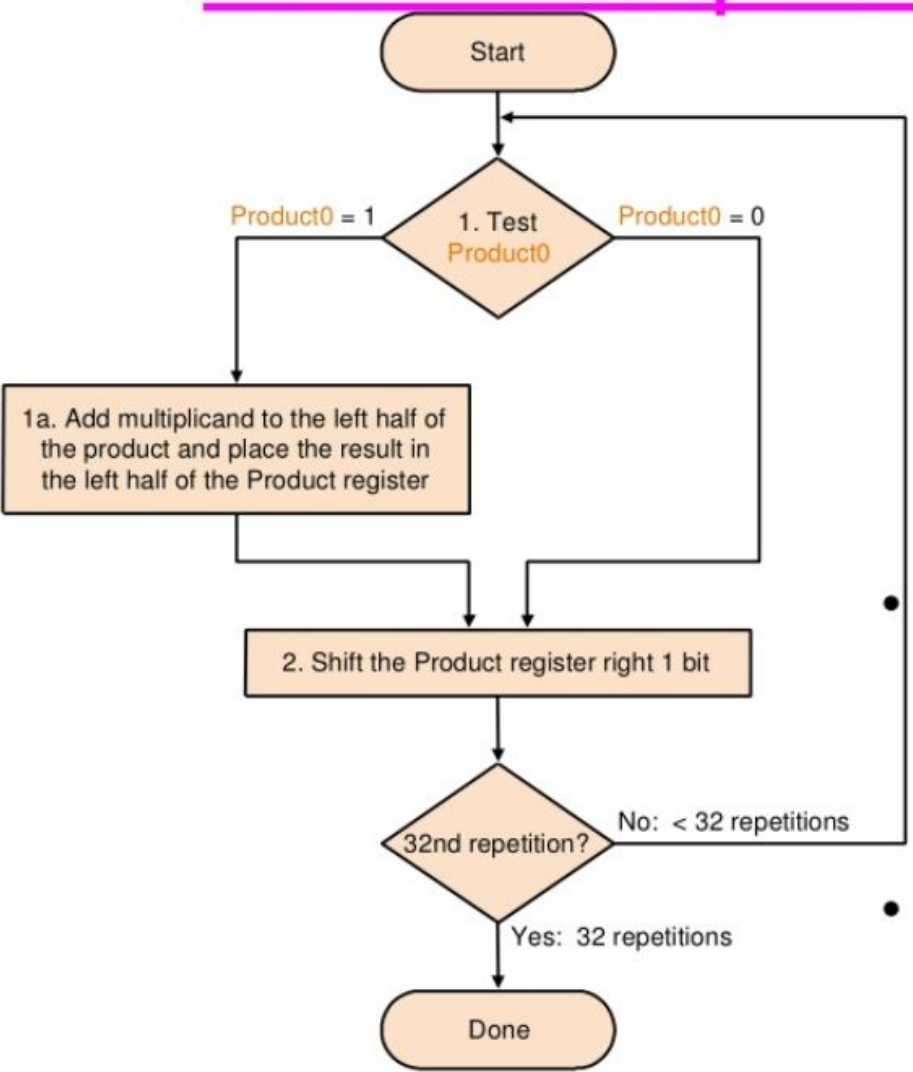
Multiply 3 x 2 ?

2 = 0010

3 = 0011

Steps	Mutiplicand	Multiplier	Product
Init	0011	0010 0010	0000 0000
2		0010 0010	0000 0000
3		0010 0001	0000 0000
1a		0010 0001	0011 0000
2		0010 0001	0001 1000
3		0010 0000	0001 1000
2		0010 0000	0000 1100
3		0010 0000	0000 1100
2		0010 0000	0000 0110
3		0010 0000	0000 0110
			= 6 (ANS)

FINAL (REFINED) VERSION OF MULTIPLICATION ALGORITHM



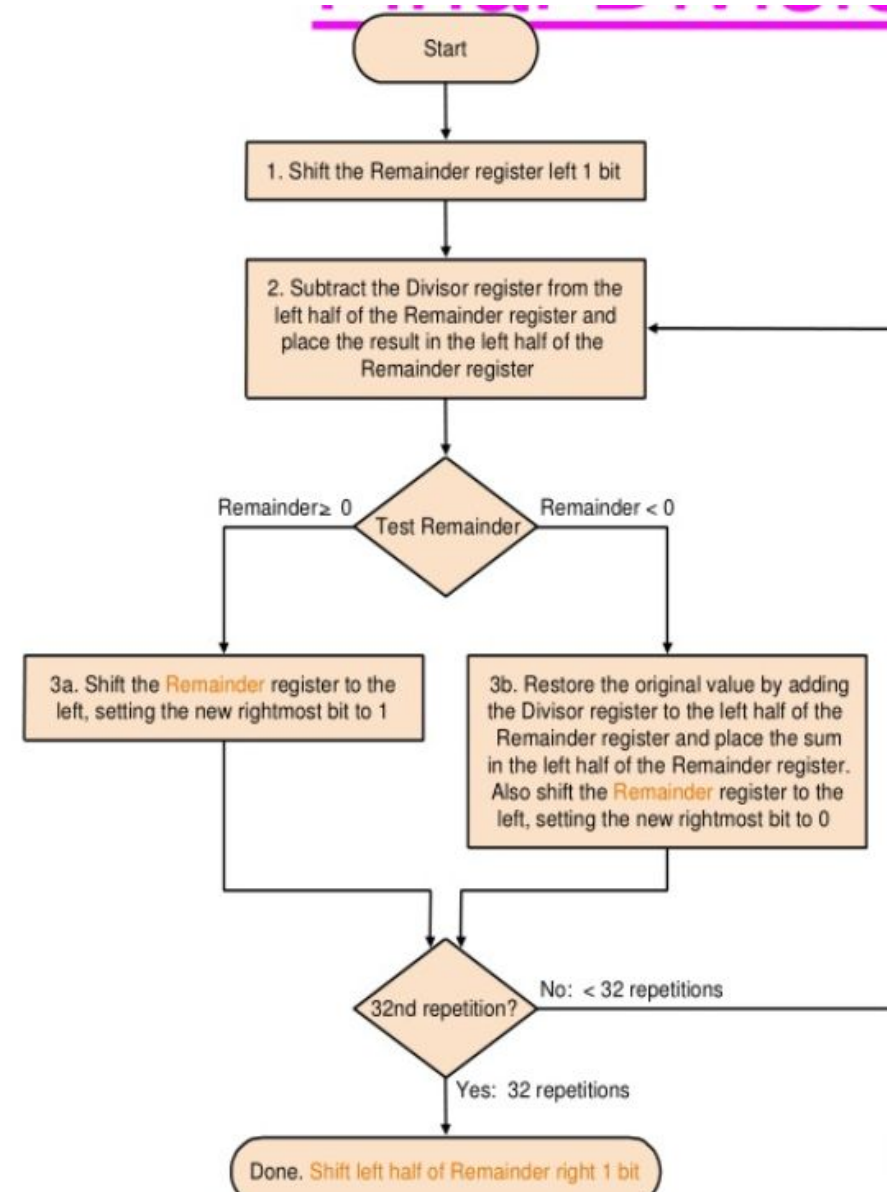
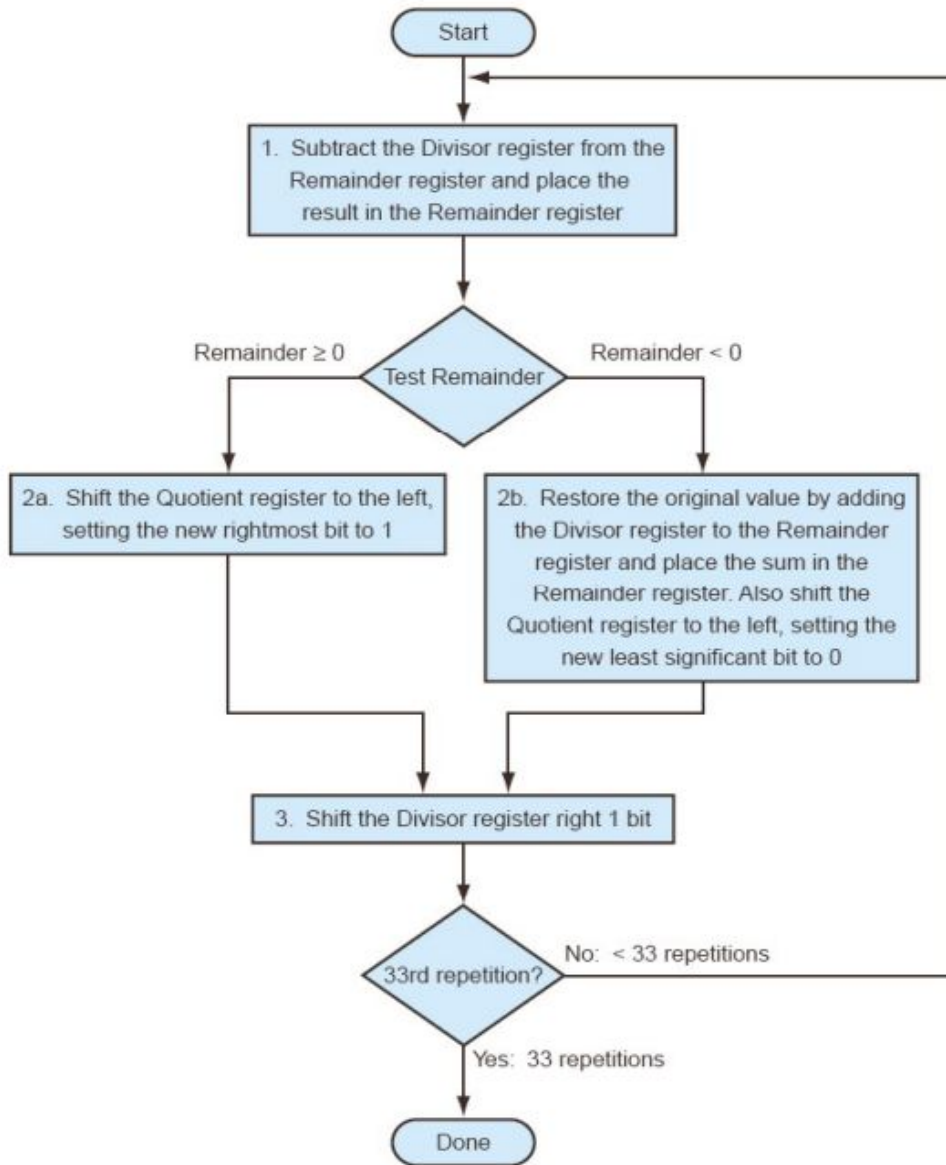
Multiply 3 x 2 ?

2 = 0010

3 = 0011

Steps	Mutiplicand	Product0	Product
Init	0011	0000 0010	0000 0010
2		0000 0010	0000 0001
1a			0011 0000
2		0000 0010	0001 1000
2		0000 0010	0000 1100
2			0000 0110 = 6 (ANS)

FIRST AND FINAL VERSION OF THE DIVISION ALGORITHM



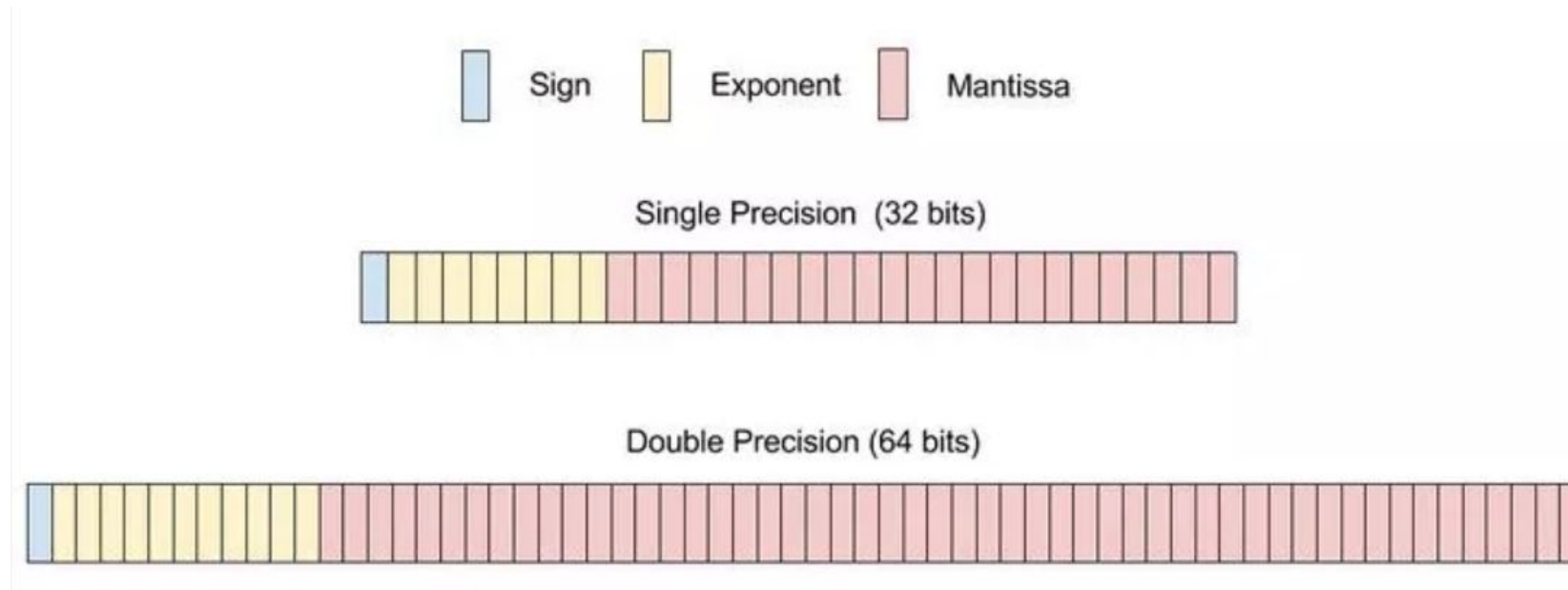
RESOURCES FOR THE ALGORITHMS

- [First Version of Multiplication Algorithm](#)
- [Final Version of the Multiplication Algorithm](#)
- [First Version of the Division Algorithm](#)
- [Final Version of the Division Algorithm](#)

Click on the titles to redirect to the resources.

FLOATING POINT REPRESENTATION

- Operations on real numbers, using formats like IEEE 754 to represent large/small decimal numbers.
- Single and Double precision of **IEEE 754**:



FLOATING POINT REPRESENTATION

- *Sign Bit*

1. The sign bit determines whether the number is positive or negative.
2. 1 bit is allocated for the sign.
3. If the sign bit is 0, the number is positive.
4. If the sign bit is 1, the number is negative.

- *Exponent*

- The exponent determines the power of 2 to which the mantissa is raised.
- It is stored using biased notation:
 1. Single precision (32-bit): 8-bit exponent, bias = 127.
 2. Double precision (64-bit): 11-bit exponent, bias = 1023.
- The exponent value is stored as: $\text{Exponent} = \text{Actual exponent} + \text{Bias}$

FLOATING POINT REPRESENTATION

- *Mantissa (Fraction)*

1. The mantissa (or fraction) represents the significant part of the number.
2. In IEEE 754, we use normalized representation, meaning the leading 1 is implied and not stored.
3. Only the fractional part after the leading 1 is stored

FLOATING POINT REPRESENTATION: EXAMPLE

- Show the IEEE 754 binary representation of the number -0.375_{10} in single and double precision.
- **Step-1:** Convert 0.375 to binary:

Mul by 2		Integer part
$0.375 * 2$	0.75	0
$0.75 * 2$	1.5	1
$0.5 * 2$	1.0	1

- So, we got 0.011 in binary of 0.375.

FLOATING POINT REPRESENTATION: EXAMPLE

- **Step-2: Normalize the Binary:**
 1. To normalize we use **implicit normalization** for accuracy:
 2. *Format:* $1.xxxx * 2^n$
 3. For 0.011, it will be $1.1 * 2^{-2}$.
- **Step-3: IEEE 754 Single Precision Representation (32-bit):**
- IEEE 754 single precision consists of:
 1. 1 bit for the sign
 2. 8 bits for the exponent
 3. 23 bits for the mantissa (fractional part after 1.)

FLOATING POINT REPRESENTATION: EXAMPLE

- Sign Bit (1 bit): Since the number (-0,375) is negative, the sign bit = 1.
- Exponent (8 bits): IEEE 754 uses a biased exponent with a bias of 127:
 1. Exponent = Actual exponent+Bias = $-2+127 = 125$.
 2. Convert 125 to binary: $125 = 01111101 = 01111101 = 01111101$.
 3. So, the 8-bit exponent = 01111101.
- Mantissa (23 bits): We take the fractional part from step-2 = 1.1 (drop the leading 1.), so: 1000000.....
- **Final IEEE 754 Single Precision Representation:**
1 01111101 10000000000000000000000
- Binary: 1011 1110 1100 0000 0000 0000 0000 0000
- Hexadecimal: 0xBE800000

FLOATING POINT REPRESENTATION: EXAMPLE

- [illegible]

FLOATING POINT REPRESENTATION: EXAMPLE

- Binary: 1011 1111 1011 1000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
- Hexadecimal: 0xBFDB800000000000 (*ANS.*)

Resources:

1. [Decimal to IEEE 754](#)
2. [IEEE 754 to Decimal](#)

LONGEST COMMON SUBSEQUENCE (LCS)

- The **Longest Common Subsequence (LCS)** problem is a classic problem in computer science and dynamic programming. Given two sequences, the goal is to find the longest subsequence that appears in both sequences in the same order (but not necessarily consecutively).
- Example: Two strings given:
X = BACDB
Y = BDCB

x \ y →	B	D	C	B
↓	0	0	0	0
B	0	↖ 1	← 1	↖ 1
A	0	↑ 1	↑ 1	↑ 1
C	0	↑ 1	↑ 1	↖ 2
D	0	↑ 1	↖ 2	↑ 2
B	0	↖ 1	↑ 2	↖ 3

⇒ BCB
LCS

COMPLEXITY ANALYSIS OF ALGORITHMS

- **Complexity Analysis of Algorithms:**

- Complexity analysis of an algorithm determines the amount of resources (time and space) an algorithm requires based on the input size. It helps in evaluating algorithm efficiency, comparing different algorithms, and optimizing performance.

- *Types of Complexity Analysis*

1. Time Complexity: Measures the number of basic operations an algorithm performs as a function of input size n . Expressed using Big-O notation.

- *Common time complexities:*

1. $O(1)$ = Constant Time = Accessing an element in an array
2. $O(n)$ = Linear Time = Traversing an array
3. $O(\log n)$ = Logarithmic Time = Binary Search
4. $O(n \log n)$ = Linearithmic Time = Merge sort, Quick sort
5. $O(n^2)$ = Quadratic Time = Bubble sort

COMPLEXITY ANALYSIS OF ALGORITHMS

- *Best, Worst, and Average Case Complexity*
 1. Best Case (Ω - Omega): Minimum time an algorithm takes for optimal input.
 2. Worst Case (O): Maximum time an algorithm takes for the worst input.
 3. Average Case (Θ - Theta): Expected runtime for random input.

COMPLEXITY ANALYSIS OF ALGORITHMS

2. Space Complexity: Measures the amount of extra memory an algorithm needs during execution.

3. Why Does Complexity Analysis Matter?

1. Helps in selecting the best algorithm for a problem.
2. Identifies bottlenecks and improves efficiency.
3. Ensures scalability for large inputs

THE END